

Data Analysis

Missing values and Dimensionality Reduction

CentraleDigitalLab@Nice

Content

Preliminaries

Missing values

ML and DL tasks

Dimensionality reduction

Preliminaries

Numbers, Probability & Data Visualization

From the nature of numbers to choosing the right chart

Two Great Branches: Real & Imaginary

A number represents a quantity, magnitude, position, or relationship. All numbers used in ordinary mathematics fall into one of two families.

Real Numbers ($\mathbb{R}$)

- Every number that sits on a single, one-dimensional number line.
- Includes negatives, zero, positives, fractions, decimals.
- Includes irrational quantities such as π .

Imaginary Numbers

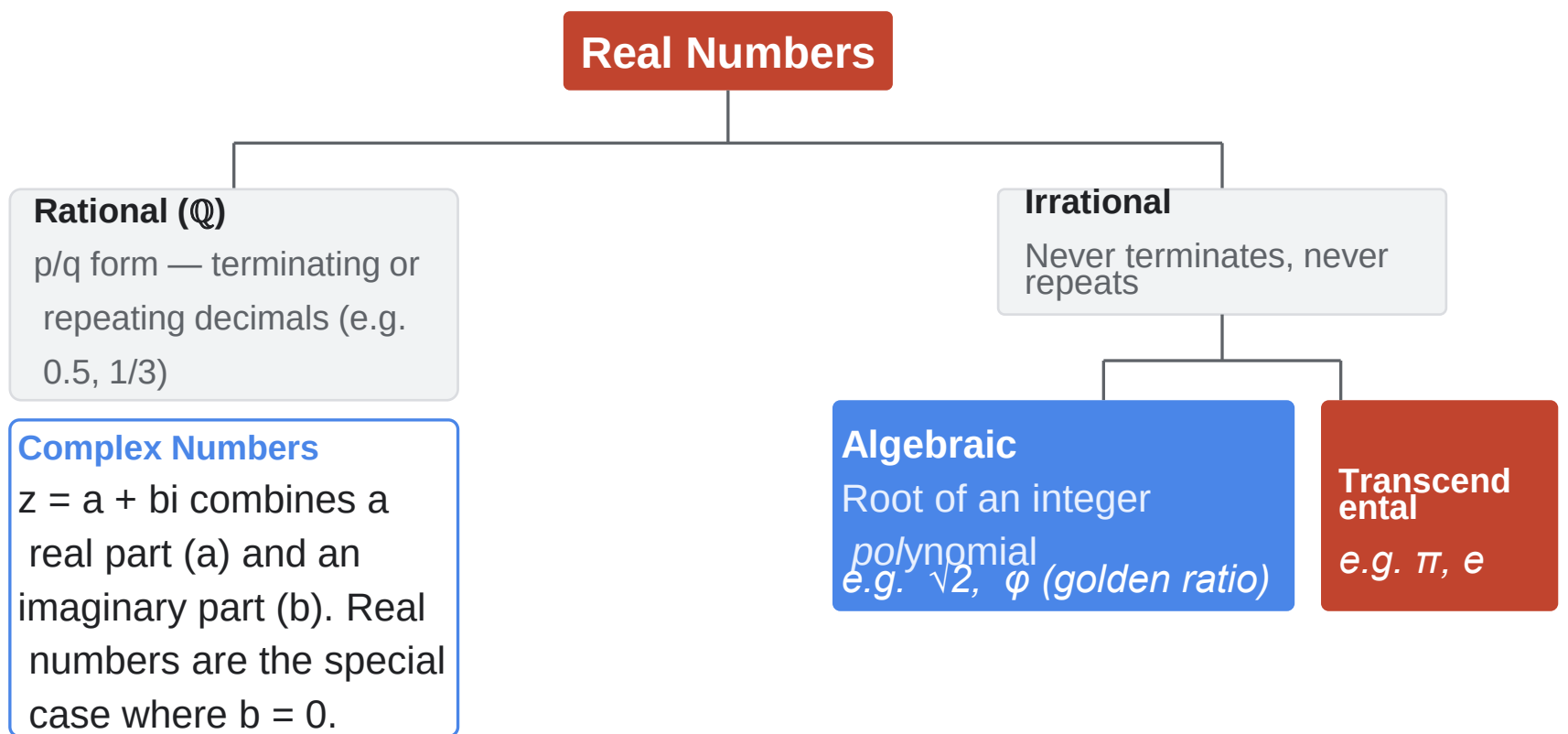
Built from i , defined by $i^2 = -1$.

No real number squared gives a negative result. A new axis, perpendicular to the real line, represents values like $\sqrt{-4} = 2i$.

A real part and an imaginary part combine into a **complex number**: $z = a + bi$. Real numbers are the special case where $b = 0$.

Inside the Real Numbers

Real numbers split by whether they can be written as a ratio of two integers — and irrational numbers split again.



Discrete vs. Continuous Sets

How “densely packed” is a set of numbers? This idea threads through probability and data visualization alike.

Discrete

Values are separate and countable, with gaps between them — list them one at a time (0, 1, 2, 3, ...). **Nothing else in the set exists** between two consecutive values.

Continuous

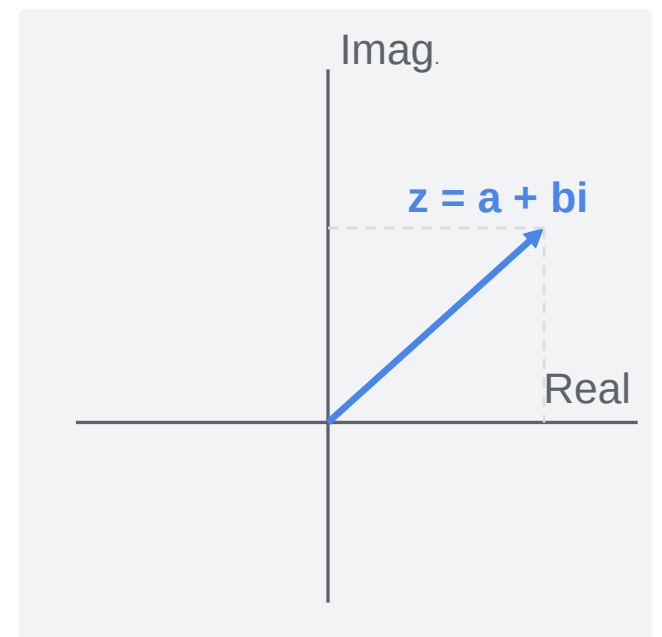
Values can take any value in a range, including every fraction and irrational in between. **There is no “next” value** — infinitely many lie between any two.

Set type	Meaning	Example
Finite	A fixed, countable number of elements that ends.	{1,2,3,4,5} — a die roll
Countably infinite	Infinite, but elements can be listed one after another, matched to 1, 2, 3, ...	All integers; all rationals
Uncountably infinite	So dense the elements cannot be listed one by one.	The reals between 0 and 1

Complex Numbers as 2D Vectors

A vector is an ordered tuple of real numbers, $v = (v_1, v_2, \dots, v_n)$, specifying a point or direction-and-magnitude in n -dimensional space.

- **$z = a + bi$ is exactly the 2D vector (a, b)**
- Adding complex numbers = adding vectors component-wise
- $|z| = \sqrt{a^2 + b^2}$ is the vector's Euclidean length
- Multiplying by i = a 90° rotation about the origin



Descriptive Statistics: Mean, Median, Mode

Before describing a full distribution, summarize a set of numbers with a single “typical value.”

Mean

$$\mu = \Sigma x / n$$

Sum divided by count
• Sensitive to outliers.

*5 setosa petal lengths:
1.4, 1.4, 1.3, 1.5, 1.4 →
mean = 1.4 cm*

Median

Middle value, sorted

Robust to outliers.

Runtimes

*90, 95, 100, 105, 300 →
median = 100 (mean
would be 138)*

Mode

Most frequent value

Works on categorical
data.

*Most common IMDb
genre in a catalog,
e.g. “Drama”*

Multidimensional data: statistics are computed column-by-column (a mean vector / centroid), or per group — e.g. *Iris setosa's mean petal length (≈1.46 cm) vs. virginica's (≈5.55 cm).*

Conditional Probability & Bayes' Theorem

Conditional Probability

The probability of A occurring given that B has already occurred.

$$P(A | B) = P(A \cap B) / P(B)$$

Example: among 100 emails, 20 are spam and 15 of those contain “free.” Given an email contains “free,” what fraction are spam?

Bayes' Theorem

Reverses a conditional probability: goes from $P(B | A)$ to $P(A | B)$ — essential when only the reverse direction is measurable.

$$P(A|B) = [P(B|A) \cdot P(A)] / P(B)$$

Example: a test is 99% accurate for a disease only 1% of people have. Bayes' theorem shows $P(\text{disease} | \text{positive test})$ is far lower than 99% — the rare prior pulls it down.

Random Variables & Why Distributions Matter

Random Variable

- Discrete: countable values, e.g. heads in 10 coin flips (0–10)
- Continuous: any value in a range, e.g. exact time until a bulb fails

PMF vs. PDF

- Discrete → PMF: assigns real probability per outcome, e.g. $P(\text{die}=3)=1/6$
- Continuous → PDF: density; probability = area under the curve over an interval

Why is a probability distribution necessary?

A mean alone tells you where data is centered — not how spread out it is or what fraction of outcomes fall in a range you care about. *Example: bolts must be 10 mm ± tolerance. Knowing only the mean diameter (10 mm) says nothing about quality. Knowing diameters follow $\text{Normal}(\mu=10, \sigma=0.05 \text{ mm})$ lets you compute exactly what fraction fall outside 9.9–10.1 mm — before manufacturing a single batch.*

Common Probability Distributions

Discrete

Binomial

$$C(n,k) \cdot p^k \cdot (1-p)^{n-k}$$

Number of successes in n independent yes/no trials.

Ex: Heads in 10 coin flips

Discrete

Poisson

$$\lambda^k \cdot e^{-\lambda} / k!$$

Count of rare events in a fixed interval, rate λ .

Ex: Emails arriving per hour

Continuous

Normal

bell curve (μ, σ)

Sum of many small independent effects (CLT).

Ex: Human height, petal length

Continuous

Student's t

heavier-tailed bell

Small-sample mean estimation, σ unknown.

Ex: 15-patient clinical trial

Matching Data Type to Chart

Data type	Example	Best charts
Continuous	Height, petal length	Histogram, density, box plot, scatter, line
Discrete (numeric)	Number of awards	Bar chart, dot plot
Categorical – Nominal	Species, genre	Bar chart, pie chart (few categories)
Categorical – Ordinal	Rating tier (G/PG/R)	Ordered bar, stacked bar
Binary	Spam / not spam	Bar chart, pie chart
Time series	Daily box-office revenue	Line chart, area chart
Text	Review text	Word cloud, word-frequency bar

Which Chart, and Why

1	Histogram	Distribution / shape of one continuous variable
2	Bar chart	Compare counts across discrete/categorical groups
3	Line chart	Trend of a continuous variable over time
4	Box plot	Compare spread/outliers across categories
5	Scatter plot	Relationship between two continuous variables
6	Pie chart	Share of a whole, few categories
7	Heatmap	Correlation across many variables at once

Case Study: The Iris Dataset

150 flowers, 3 species (setosa, versicolor, virginica), 50 each — 4 continuous measurements + 1 categorical label.

Column	Type	Recommended visualization
Sepal / petal length & width	Continuous	Histogram / density per feature; scatter (e.g. petal length vs. width) reveals species clusters
Species	Categorical (nominal)	Bar chart of counts per species
Measurement × species	Continuous grouped by category	Box plot per species; scatter colored by species

Best single chart for Iris: a scatter plot of petal length vs. petal width, colored by species — the three species separate into visibly distinct clusters, making it the clearest way to see how well the measurements predict species.

Case Study: The IMDb Dataset

A mix of data types: title (text), genre (categorical), year (temporal), runtime & rating (continuous), votes/revenue (skewed), review text + sentiment (binary).

Column	Type	Recommended visualization
Genre	Categorical (nominal, multi-label)	Bar chart of genre frequency
Release year	Discrete / temporal	Line chart of count or avg. rating per year
Runtime	Continuous	Histogram
Votes / revenue	Continuous, skewed	Histogram on log scale; scatter vs. rating
Sentiment label	Binary	Bar chart of positive vs. negative
Review text	Text	Word cloud; review-length histogram

Key Takeaways

- Numbers split into real / imaginary, then rational / irrational, then algebraic / transcendental.
- Discrete vs. continuous is one idea that recurs in sets, random variables, and data columns alike.
- A probability distribution — not just a mean — is what lets you compute risk and make real decisions.
- The right chart follows directly from a column's data type.

Missing Values

Handling missing values is a critical step in preparing data for machine learning models since many **models cannot handle missing values** out of the box.

Missing Values

Handling missing values is a critical step in preparing data for machine learning models since many **models cannot handle missing values** out of the box.

We have three types of missing values:

- Missing Completely At Random (MCAR)
- Missing At Random (MAR)
- Missing Not At Random (MNAR)

Missing Completely At Random (MCAR)

Definition: Data is said to be Missing Completely At Random (MCAR) when the **probability of missingness is the same for all observations**. In other words, the missingness is independent of both observed and unobserved data.

Missing Completely At Random (MCAR)

Definition: Data is said to be Missing Completely At Random (MCAR) when the **probability of missingness is the same for all observations**. In other words, the missingness is independent of both observed and unobserved data.

Example: Suppose you have a dataset of student test scores and due to a clerical error, random pages of the data are lost. The **missing data** is likely MCAR as it **has no relationship with any other data**.

Missing Completely At Random (MCAR)

Definition: Data is said to be Missing Completely At Random (MCAR) when the **probability of missingness is the same for all observations**. In other words, the missingness is independent of both observed and unobserved data.

Solution: **Simple Imputer**, such as using the **mean**, **median**, or **most frequent value** of the feature. You can also use a constant value.

Missing At Random (MAR)

Definition: Data is said to be Missing At Random (MAR) if the probability of missingness is systematic and **can be predicted by other observed data, but not by the missing data itself.**

Example: In a health survey, suppose females are less likely to report their weight. The missingness of weight data is systematic and related to the observed gender data but is not related to the weight data itself.

Missing At Random (MAR)

Definition: Data is said to be Missing At Random (MAR) if the probability of missingness is systematic and **can be predicted by other observed data, but not by the missing data itself.**

Missing At Random (MAR)

Definition: Data is said to be Missing At Random (MAR) if the probability of missingness is systematic and **can be predicted by other observed data, but not by the missing data itself.**

Solution: K-Nearest Neighbors (KNN) Imputation, Iterative Imputer, or Regression Models. The other variables in the dataset predict the missing variable.

Missing At Random (MAR)

Definition: Data is said to be Missing At Random (MAR) if the probability of missingness is systematic and **can be predicted by other observed data, but not by the missing data itself**.

K-Nearest Neighbors (KNN) Imputation works as follows:

1. Select the k nearest neighbors based on a distance metric.
2. Calculate the mean, median, mode, or weighted average value of the K neighbours to fill the missing values.

Missing At Random (MAR)

Definition: Data is said to be Missing At Random (MAR) if the probability of missingness is systematic and **can be predicted by other observed data, but not by the missing data itself.**

Iterative Imputer works as follows:

1. Fill in missing values with a simple imputation (mean, median, etc)
2. Create a **regression model to predict** missing values in the selected feature based on the values of the other features.
3. **Impute** missing values **with the regressor**.
4. **Iterate** through this process multiple times in a round-robin fashion **until the imputation values converge**.

Missing Not At Random (MNAR)

Definition: Data is Missing Not At Random (MNAR) if the **probability of missingness is related to the missing data itself**, even when controlling for other observed variables.

Example: In a mental health survey, individuals with severe depression might be less likely to respond to questions about mental health. The **missingness** in responses is **related to the unobserved** mental health data.

Missing Not At Random (MNAR)

Definition: Data is Missing Not At Random (MNAR) if the **probability of missingness is related to the missing data itself**, even when controlling for other observed variables.

Solution: Advanced Model-Based Imputation such as Random Forests or Bayesian Networks which can capture complex relationships in data and might reveal the hidden structure causing the missingness.

Demo with notebook

01__missing_values.ipynb

Machine Learning Tasks & Deep Learning

Which data fits which task, and how deep learning grew out of classical ML

What Are Regression & Classification?

Regression

Predicting a **continuous numeric output** from input features. Error is measured by how far off, on average, the numeric predictions are (e.g. mean squared error).

Example

Predicting a movie's box-office revenue from budget, genre, and runtime.

Classification

Predicting a **categorical output** — assigning an input to one of a fixed set of discrete classes. Error is measured by accuracy, precision/recall, or similar.

Example

Predicting an iris flower's species from its measurements, or a review's sentiment.

Matching Data Type to ML Task

Target type	ML task	Dataset example
Continuous numeric	Regression	Predicting IMDb box-office revenue or runtime
Categorical, ≥ 2 classes	Classification	Predicting iris species from measurements
Binary categorical	Binary classification	Predicting IMDb review sentiment (pos/neg)
Ordinal categorical	Ordinal regression (hybrid)	Predicting a movie's star rating tier
Free text	Feature extraction, then classify/regress	Classifying IMDb review sentiment from raw text

In short: *Iris — continuous measurements + categorical species — is a textbook classification dataset. IMDb can serve either purpose: revenue/runtime → regression; genre/sentiment → classification.*

Tasks With No Labels: Unsupervised Learning

Regression and classification are the two most common supervised tasks. Other ML tasks

differ in structure — starting with tasks that have no labels at all.

1 Clustering

Group similar points together without predefined classes.

2 Dimensionality reduction

Compress high-dimensional data while preserving structure (PCA, t-SNE, UMAP).

3 Density estimation

Learn the underlying probability distribution of the data itself.

4 Anomaly detection

Flag points that don't fit the learned distribution.

Tasks Built From Regression + Classification

Many tasks combine both atoms at once, or apply them repeatedly across a structured output.

Ranking

Order items by relevance

Search results ordering

Structured prediction

Sequence, tree, or set of objects, not one value

Translation (per-token classification); object detection (box regression + class)

Generative modeling

Produce new samples resembling the training distribution

A diffusion model's training step is a regression problem (predict the noise added)

Different Reward Structures

Reinforcement Learning

Learn a **policy** (sequence of actions) that maximizes reward through trial and error, not a fixed input→output mapping.

- In actor-critic RL: the critic is regression (predicts expected future reward); the actor is classification (discrete actions) or regression (continuous control).
- The hard part — crediting a past action for a later reward — is sequential credit assignment, not a regression/classification difficulty.

Self-Supervised Learning

Learn useful features from unlabeled data by solving an **auxiliary task** — e.g. predicting a masked word.

Often reframed internally as classification: pick the correct item out of many candidates (contrastive learning).

Two tasks can share the same regression/classification atoms and still be fundamentally different problems.

ML vs. Deep Learning

Machine Learning

Algorithms that learn patterns from data instead of hand-written rules. Many rely on **hand-engineered features** chosen by a person.

Deep Learning

A subset of ML built from neural networks with multiple stacked layers, **learning its own intermediate representations** directly from raw data.

	Classical ML	Deep Learning
Features	Usually hand-engineered	Learned automatically, layer by layer
Data needs	Works well on small/medium data	Usually needs large data
Interpretability	Often high	Usually low (“black box”)
Structure	Varies widely by algorithm	Stacked linear transforms + nonlinearity

Classical ML Algorithms Hiding Inside a Neuron

The boundary isn't a hard line — several classical ML algorithms are literally a one-layer neural network.



Linear Regression

A single neuron, no hidden layer, identity activation.

computes $w \cdot x + b$



Logistic Regression

A single neuron with a sigmoid activation on that same linear combination.

one-layer NN for binary classification



The Perceptron (1958)

A single neuron with a step-function activation.

ancestor of both, and of modern neural nets

When Does It Become Deep Learning?

The transition is a gradual stacking process, not a single event.



The conceptual trigger: a model crosses from classical ML into deep learning the moment it stops using features supplied by a person and starts learning its own intermediate representations through stacked, nonlinear, trainable transformations.

Key Takeaways

- Regression predicts a continuous number; classification predicts a discrete category.
- Most other ML tasks build on these two atoms, but differ in supervision and output structure.
- Deep learning is ML built from stacked neural network layers that learn their own features.
- Linear and logistic regression are literally one-layer neural networks — DL is a stacking process, not a hard boundary.

Dimensionality Reduction

**Reduce the number of
columns or variables in our
dataset**



**Preserve as much
information as possible**

Dimensionality Reduction

Let's express the data set as a **matrix X with n rows and m columns**. Each row is a vector x_i that inhabits a mathematical space with m dimensions. **Each dimension intuitively corresponds to a column**.

$$X \in \mathbb{R}^{n \times m}; x_i \in \mathbb{R}^m$$

Dimensionality Reduction

Let's express the data set as a **matrix X with n rows and m columns**. Each row is a vector x_i that inhabits a mathematical space with m dimensions. **Each dimension intuitively corresponds to a column**.

$$X \in \mathbb{R}^{n \times m}; x_i \in \mathbb{R}^m$$

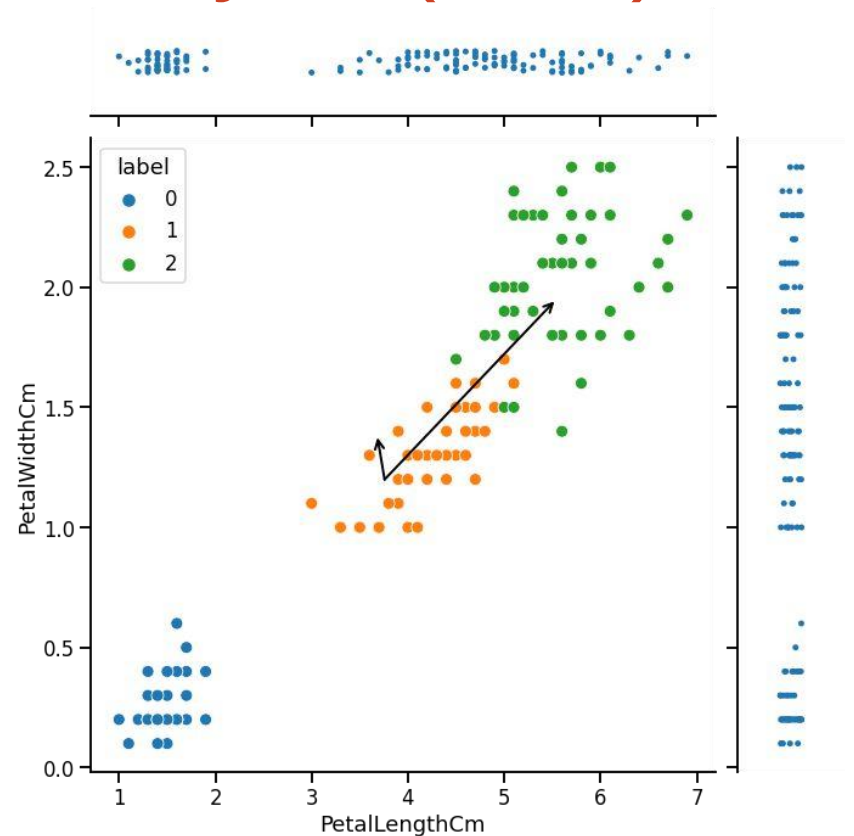
We want to obtain a **new matrix Z** that has the **same number of rows**, but a number of **columns d much smaller than m** .

$$Z \in \mathbb{R}^{n \times d}; d \ll m$$

Principal Components Analysis (PCA)

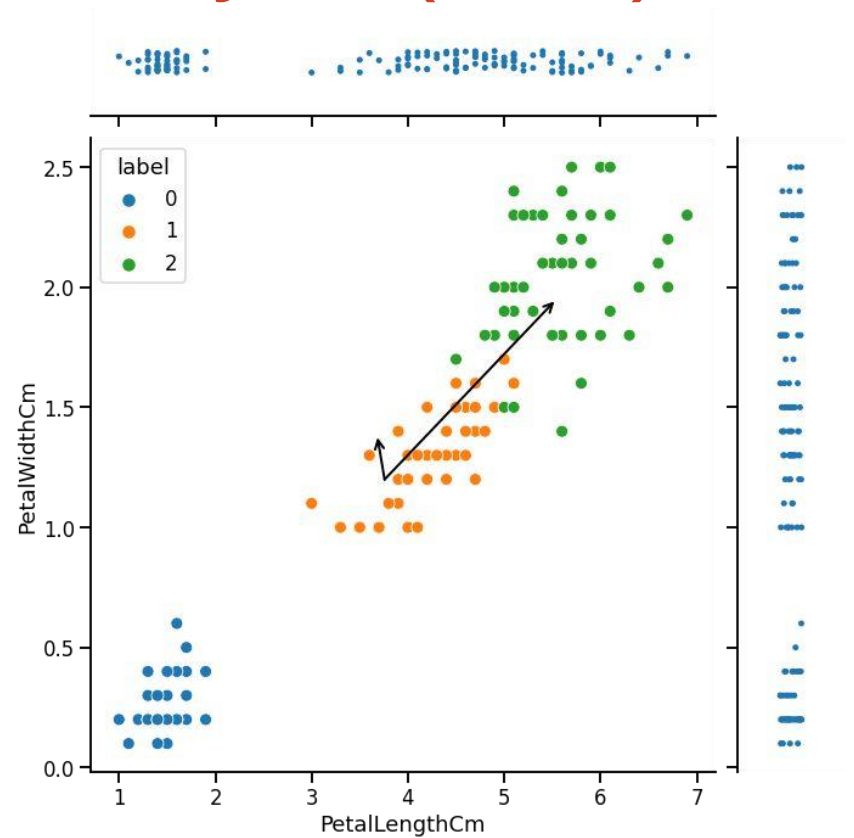
PCA is a statistical procedure that **transforms a set of observations** of possibly correlated variables into a set of values of **linearly uncorrelated variables** called principal components.

This technique is used for dimensionality reduction, visualization, and noise reduction.



Principal Components Analysis (PCA)

PCA is a statistical procedure that **transforms a set of observations** of possibly correlated variables into a set of values of **linearly uncorrelated variables** called principal components.



Principal Components Analysis (PCA)

Pros:

- **Dimensionality Reduction**: PCA allows for the reduction of the dataset dimensionality while retaining the most important information.
- **Improves Model Performance**: By eliminating redundant features, PCA can improve the performance of ML models.
- **Noise Reduction**: PCA can help in noise reduction by isolating and discarding lower-variance components.
- **Visualization**: Helps in visualizing high-dimensional data in a 2D or 3D space which can help in understanding the structure and relations in the data.

Principal Components Analysis (PCA)

Cons:

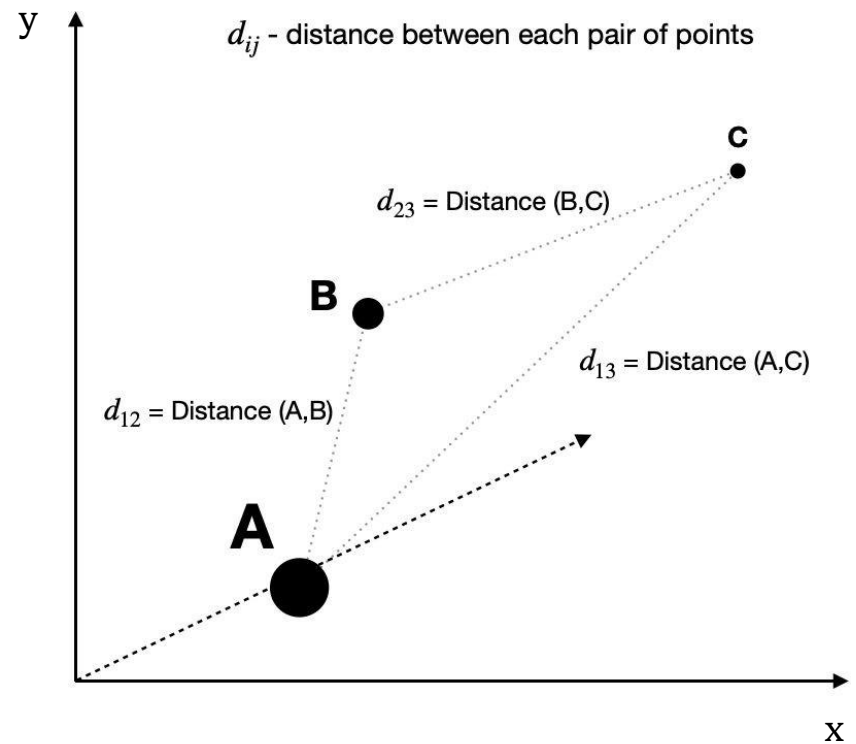
- **Loss of Interpretability**: The principal components are linear combinations of the original variables, which may result in loss of interpretability.
- **Assumes Linearity**: Assumes that the data structure is linear, which might not always be the case.
- **Sensitive to Scaling**: PCA is sensitive to the scaling of variables, so data needs to be properly scaled (like standardization) before applying PCA.

Demo with notebook
02__pca_reduction.ipynb

MDS

Steps used by MDS algorithm

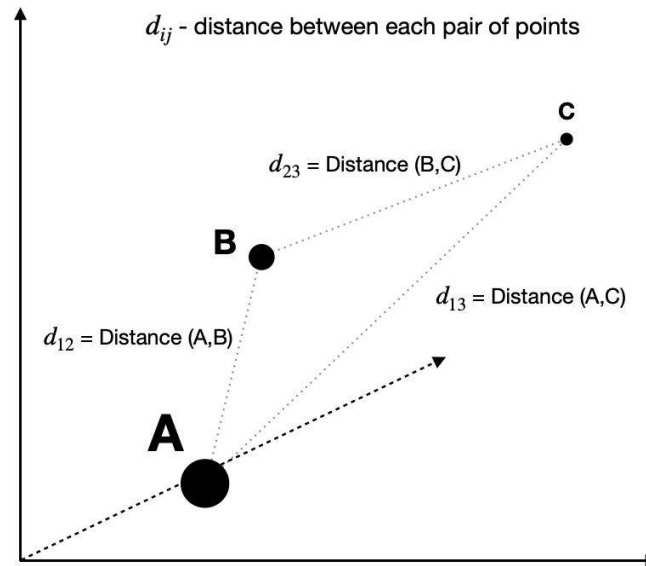
1. The algorithm calculates **distances between each pair of points**.



MDS

Steps used by MDS algorithm

1. The algorithm calculates **distances between each pair of points**.
2. With the original distances known, the algorithm attempts to **solve an optimization problem** by finding a set of coordinates in a lower-dimensional space that minimizes the value of Stress



$$Stress_D(x_1, x_2, \dots, x_N) = \sqrt{\sum_{i \neq j=1, \dots, N} (d_{ij} - ||x_i - x_j||)^2}$$

The goal of the algorithm is to minimize the value of stress.

Where $x_1, \dots, x_N$ are data points with their new set of coordinates in lower dimensional space.

d_{ij} is the actual distance we have calculated between the two corresponding data points in their original dimensional space.

$||x_i - x_j||$ is the distance between the two corresponding data points in their lower dimensional space.

The closer the value of $||x_i - x_j||$ is to d_{ij} the smaller will be the value of stress.

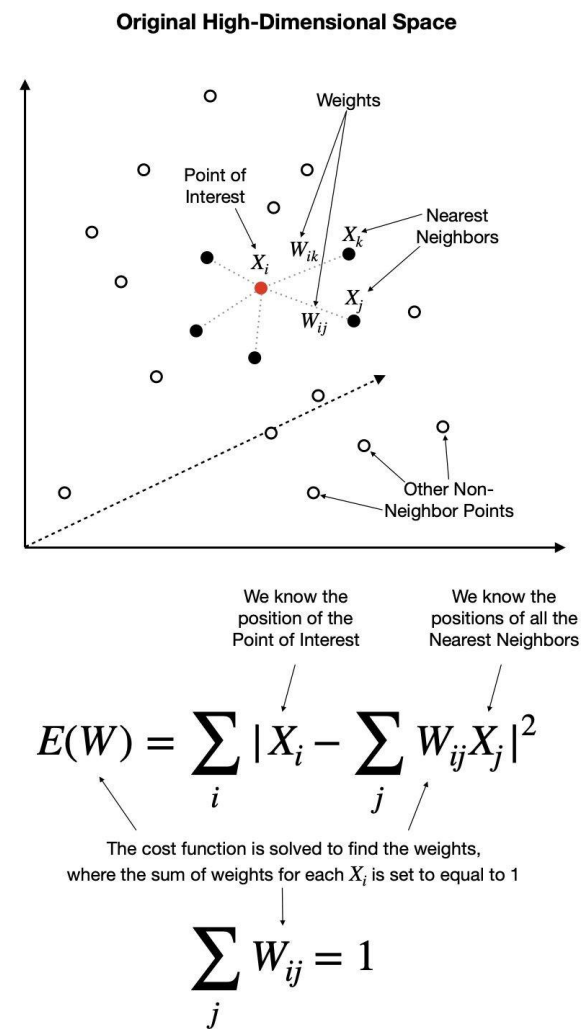
Demo with notebook

03__mds_reduction.ipynb

LLE

LLE is a **nonlinear technique** that attempts to **preserve local neighborhood relationships** in the lower-dimensional space.

It assumes that each data point and its **neighbors lie on or close to a locally linear patch** of manifold and tries to maintain these local linear relations in the reduced space.

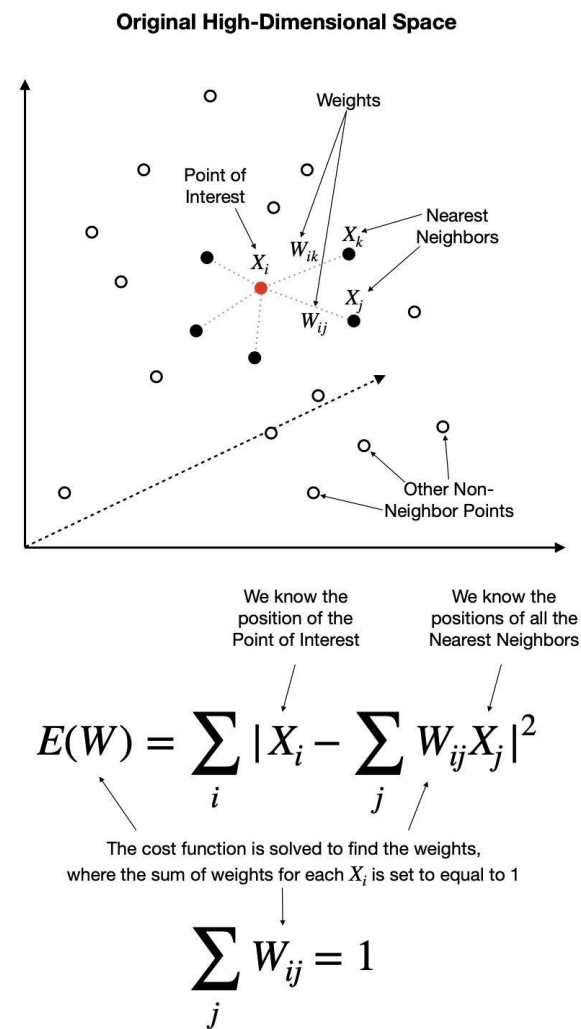


LLE

Steps used by LLE algorithm

1. Use a KNN approach to **find the k nearest neighbors** of every data point. Here, “k” is an arbitrary number of neighbors that you can specify within model hyperparameters.

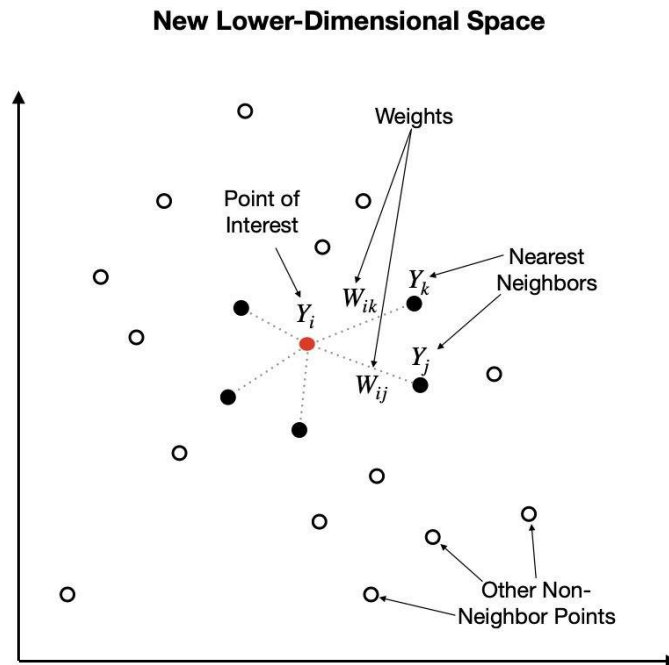
2. **Construct a weight matrix** where every point has its weights determined by minimizing the error of a cost function E. Every point is a linear combination of its neighbors, which means that **weights for non-neighbors are 0**.



LLE

Steps used by LLE algorithm

3. **Find the positions** of all the points in the **new lower-dimensional embedding** by minimizing the cost function C . Here we use weights (W) from step 2 and solve for Y .



We know the weights from the previous step

$$C(Y) = \sum_i |Y_i - \sum_j W_{ij} Y_j|^2$$

The cost function is solved to find the positions of Y_i and its neighbors in the new lower-dimensional space using weights from the previous step.

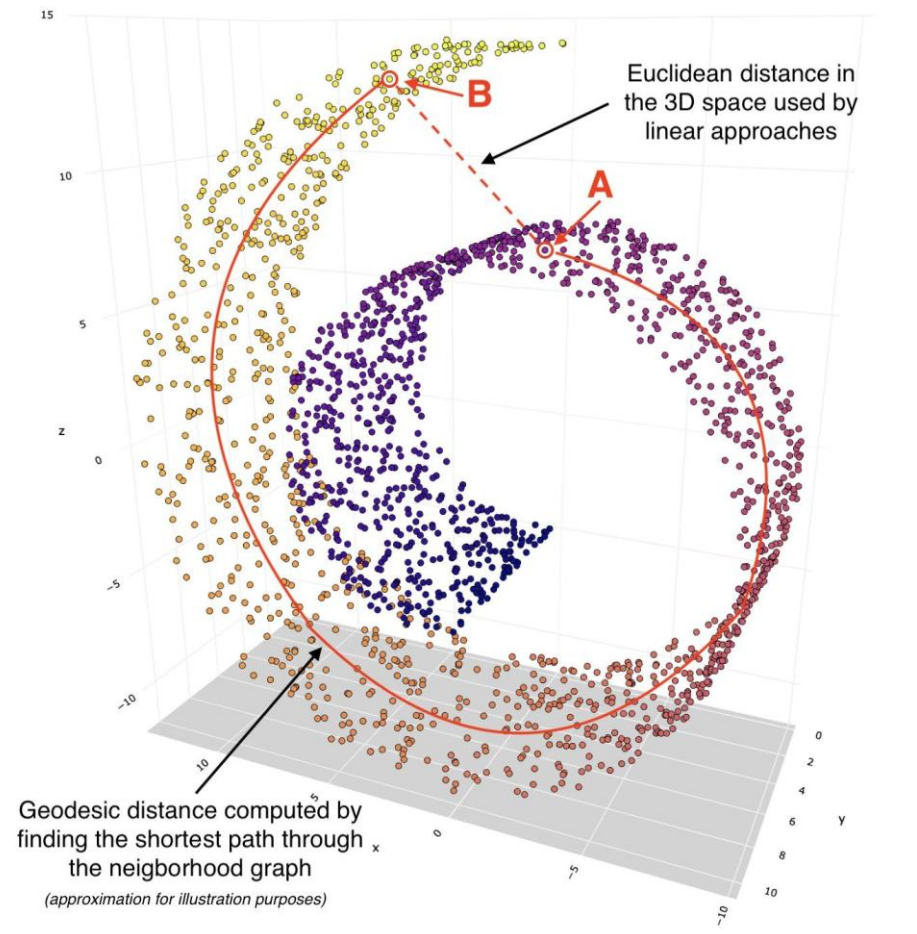
Demo with notebook
04__lle_reduction.ipynb

Isomap

Steps used by Isomap algorithm

1. Use a KNN approach to **find the k nearest neighbors** of every data point. Here, “ k ” is an arbitrary number of neighbors that you can specify within model hyperparameters.
2. Once the neighbors are found, **construct the neighborhood graph** where points are connected to each other if they are each other’s neighbors. Data points that are not neighbors remain unconnected.

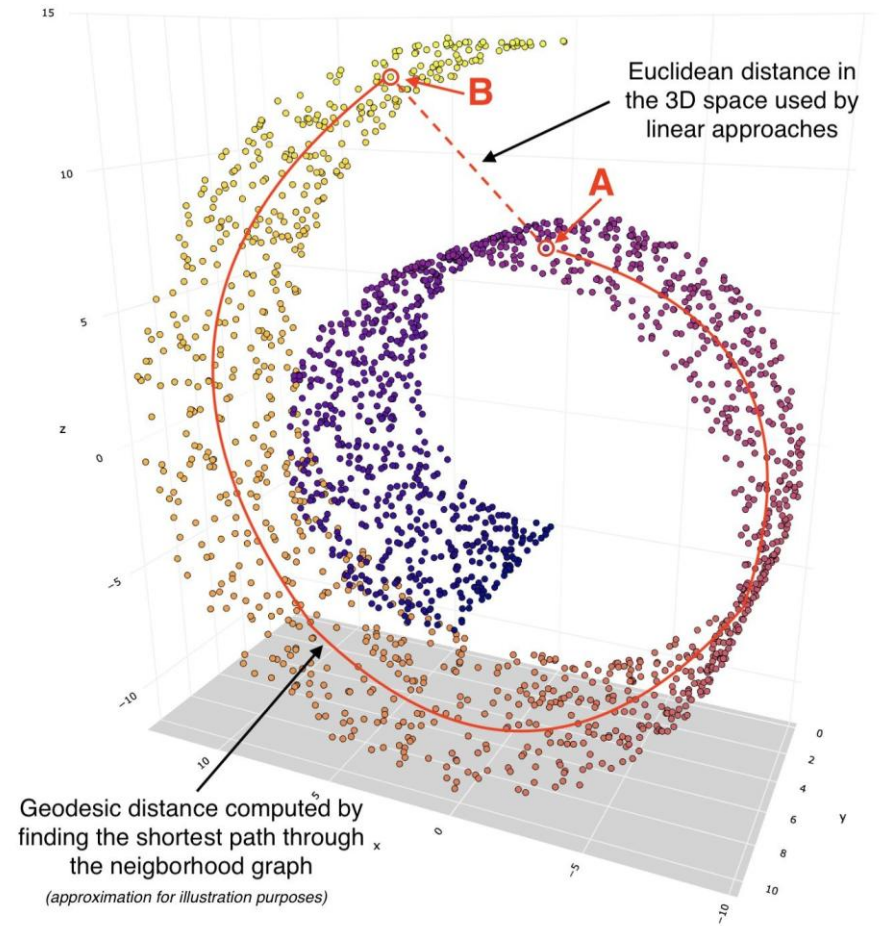
(Similar to LLE)



Isomap

Steps used by Isomap algorithm

3. **Compute the shortest path** between each pair of data points (nodes) (Dijkstra's algorithm). Note, this step is also commonly described as finding a **geodesic distance** between points.
4. **Use MDS to compute lower-dimensional embedding**. Given distances between each pair of points are known, MDS places each object into the N-dimensional space (hyperparameter) such that the between-point distances are preserved as well as possible.



Demo with notebook

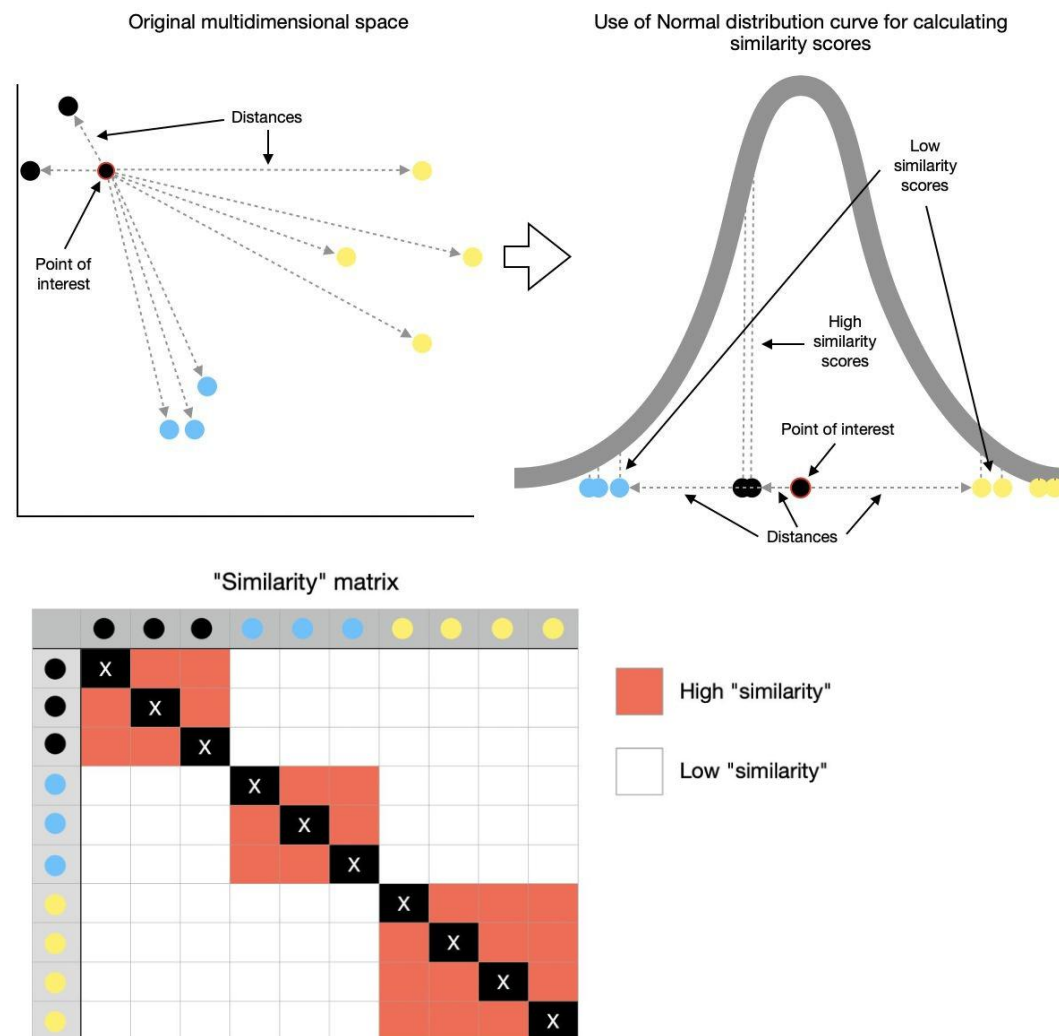
05__isomap_reduction.ipynb

T-SNE

Steps used by t-sne algorithm

1. Determine the “similarity” of points based on distances between them. Nearby points are considered “similar,” while distant ones are considered “dissimilar.”

Measuring distances between the point of interest and other points and then placing them on a Normal curve. It does this for every point, applying some scaling to account for variations in the density of different regions.



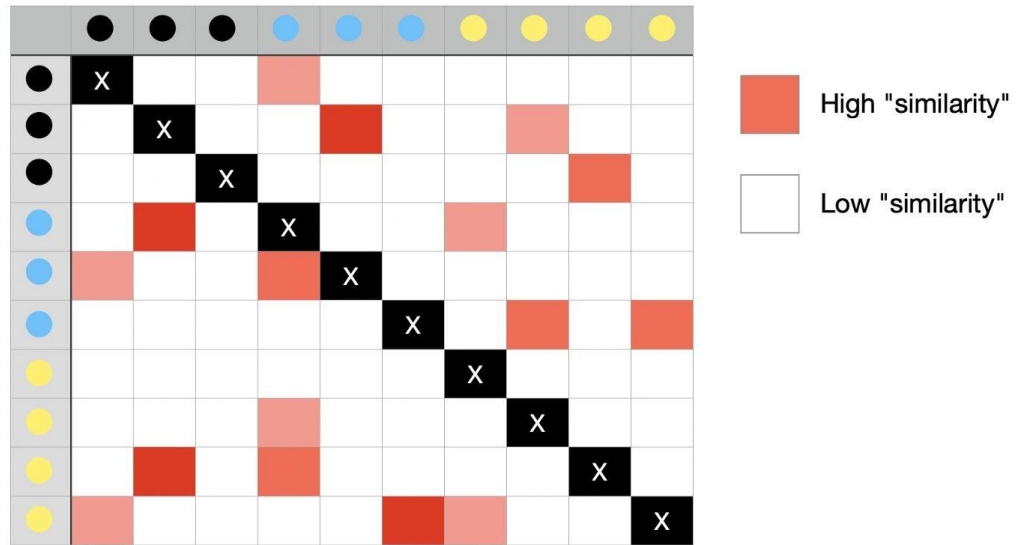
T-SNE

Steps used by t-sne algorithm

2. Randomly **map all the points onto a lower-dimensional space** and calculates “similarities” between points. One difference, though, this time, the algorithm uses **t-distribution** instead of Normal distribution.

3. Make the new “similarity” matrix look like the original one by using an iterative approach (Kullback-Leibler). With each iteration, points **move towards their “closest neighbors”** from the original higher-dimensional space and away from the distant ones.

Example of a new "Similarity" matrix

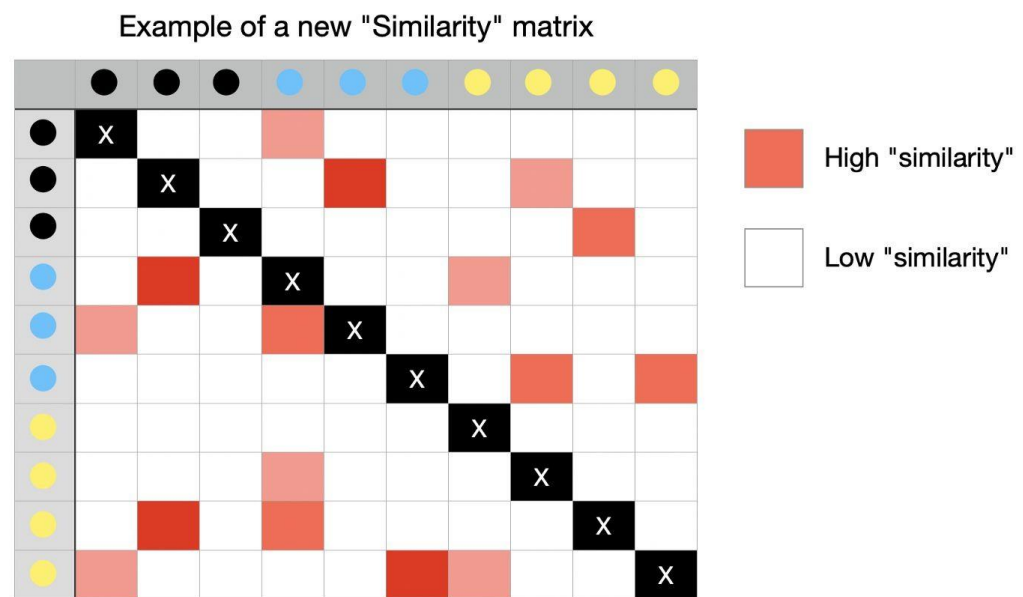


T-SNE

Steps used by t-sne algorithm

2. Randomly **map all the points onto a lower-dimensional space** and calculates “similarities” between points. One difference, though, this time, the algorithm uses **t-distribution** instead of Normal distribution.

3. Make the new “similarity” matrix look like the original one by using an iterative approach (Kullback-Leibler). With each iteration, points **move towards their “closest neighbors”** from the original higher-dimensional space and away from the distant ones.



<https://distill.pub/2016/misread-tsne/>

Demo with notebook

06__tsne_reduction.ipynb